

Actividad de Evaluación 2 · Paradigmas y Lenguajes de Programación II

Simulador de procesos y memoria con POO
Propuesta intercátedra con Sistemas Operativos

Carrera	Ingeniería en Sistemas de Información · Universidad de la Cuenca del Plata
Alumno	Medina, Gabriel
DNI	43.746.559
Docentes	Fernandez, José Alejandro · Stacciuoli, S. Romina
Repositorio	https://github.com/Gabimedina00/simulador-procesos-memoria
Commit o tag evaluado	v1.0-AE2 (tag de la versión entregada)
Fecha	06/10/2026

1. Introducción y objetivo

En este informe explico cómo diseñé y construí una biblioteca de clases en TypeScript que simula la administración de procesos, memoria y CPU de un sistema operativo. El trabajo es intercátedra: desde Sistemas Operativos se evalúa que el comportamiento simulado sea correcto, y desde Paradigmas II se evalúa cómo está construido el código.

Mi objetivo fue aplicar la programación orientada a objetos respetando lo que pide la cátedra: interfaces para los comportamientos, doble encapsulamiento, herencia sólo donde existe una relación "es un", polimorfismo sin condicionales por tipo, composición para armar el simulador y principios SOLID. Desarrollé con TDD (primero el test, después el código) y subí el trabajo en commits por requerimiento.

El resultado es una biblioteca sin main ni consola cuyo funcionamiento se demuestra únicamente con pruebas automatizadas: 82 tests, todos aprobados, con 99,6% de cobertura de líneas. GitHub Actions verifica en cada push que el código compile y que todas las pruebas pasen.

2. Escenario del simulador

El escenario es un sistema operativo simplificado con una sola CPU y una memoria principal que se reparte de forma contigua. Los procesos llegan, piden memoria, esperan su turno para usar la CPU, pueden bloquearse por una operación de Entrada/Salida y, al terminar, liberan sus recursos. El tiempo no se mide con el reloj real sino en ticks discretos que avanzan solo cuando se le pide al simulador, por lo que cada ejecución es reproducible.

La simulación empieza en el tick 0, con la memoria vacía y la CPU libre. Como configuración de

DNI: 43746559

Ingeniería en Sistemas de Información · Universidad de la Cuenca del Plata

AE2 06/10/2026

referencia uso 1024 KB de memoria y un quantum de 2 ticks, ambos configurables. Como la biblioteca no tiene main, los escenarios se arman y se verifican desde los tests.

3. Descripción general de la solución

Funcionalidad	Clase responsable
Crear procesos validando PID, memoria y CPU; programar un evento de E/S	Proceso
Controlar el ciclo de vida con seis estados y solo transiciones válidas	Proceso, EstadoProceso
Representar tramos de memoria libres u ocupados, dividirlos y fusionarlos	BloqueMemoria
Elegir el hueco con First-Fit, Best-Fit o Worst-Fit	FirstFit, BestFit, WorstFit (PoliticaAsignacionBase)
Asignar memoria contigua y liberarla con coalescencia	GestorMemoria
Planificar la CPU con Round Robin y contar cambios de contexto	PlanificadorRoundRobin
Calcular las métricas de la consigna	CalculadoraMetricas
Registrar procesos, coordinar las cuatro fases del tick y exponer el estado	Simulador

Quedan fuera del alcance la interfaz gráfica o de consola, la compactación de memoria, varias CPU y otros algoritmos de planificación, porque la consigna no los pide.

3.1 Entorno y herramientas

Herramienta	Versión	Uso
TypeScript	7.0	Lenguaje del proyecto, en modo strict
Node.js y npm	24	Entorno de ejecución, dependencias y scripts
Vitest	5.0	Framework de tests
@vitest/coverage-v8	5.0	Medición de cobertura
Git y GitHub	-	Control de versiones y repositorio remoto
GitHub Actions	-	Integración continua
Visual Studio Code y Git Bash	-	Editor y terminal en Windows

Comandos para reproducir todo desde la raíz del repositorio: **npm ci** (instala las dependencias exactas), **npm test** (corre las pruebas), **npm run coverage** (pruebas con reporte de cobertura en coverage/) y **npm run typecheck** (verifica los tipos).

4. Diseño y responsabilidad de las

clases

Distribuí el comportamiento para que cada responsabilidad del sistema operativo quede en una clase distinta y ninguna concentre toda la lógica. El código de producción está en src/, organizado en cuatro paquetes (procesos, memoria, planificación y simulador), y los tests en tests/ con la misma estructura.

4.1 Clases, interfaces y responsabilidades

Clase	Interfaz	Responsabilidad
Proceso	IProceso	Datos del proceso, transiciones de estado válidas, CPU restante, quantum consumido y E/S
BloqueMemoria	IBloqueMemoria	Tramo contiguo con inicio, tamaño y PID o libre; sabe dividirse y absorber a su vecino
PoliticaAsignacionBase (abstracta)	IPoliticaAsignacion	Filtro común de bloques libres y suficientes
FirstFit, BestFit, WorstFit	(heredan de la base)	Criterio para elegir entre los candidatos
GestorMemoria	IGestorMemoria	Asignar, dividir, liberar y fusionar bloques preservando las invariantes
PlanificadorRoundRobin	IPlanificador	Cola FIFO de Listos, CPU, quantum y cambios de contexto
CalculadoraMetricas	ICalculadoraMetricas	Fórmulas de ocupación, utilización de CPU y fragmentación externa
Simulador	ISimulador	Registrar procesos, coordinar las cuatro fases del tick y exponer copias del estado

Además hay dos enumeraciones (EstadoProceso con los seis estados y EventoCpu con lo que pasó en la CPU en un tick) y tipos de sólo lectura para las consultas: VistaProceso, VistaBloque, Metricas, ResultadoCpu, ResultadoTick y ConfiguracionSimulacion.

4.2 Relaciones entre clases

Relación	Tipo UML	Desde	Hacia
Implementa	Realización (línea punteada, triángulo vacío)	Cada clase	Su interfaz
Hereda	Generalización (línea llena, triángulo vacío)	FirstFit, BestFit, WorstFit	PoliticaAsignacionBase
Tiene (crea y contiene)	Composición (rombo)	Simulador	GestorMemoria,

DNI: 43746559

Ingeniería en Sistemas de Información · Universidad de la Cuenca del Plata

AE2 06/10/2026

	lleno)		PlanificadorRoundRobin, CalculadoraMetricas, Proceso (0..*)
Tiene (crea y contiene)	Composición	GestorMemoria	BloqueMemoria (1..*)
Tiene (recibe por constructor)	Agregación (rombo vacío)	GestorMemoria	IPoliticaAsignacion (1)
Mantiene una cola de	Asociación (0..*)	PlanificadorRoundRobin	Proceso
Usa	Dependencia (línea punteada, flecha)	Proceso, PlanificadorRoundRobin	EstadoProceso, EventoCpu

5. Decisiones de programación orientada a objetos

5.1 Interfaces y abstracción

Cada comportamiento tiene una interfaz que declara qué se le puede pedir a la clase, sin decir cómo lo hace: no lleva cuerpo. Las clases que colaboran se conocen a través de esas interfaces: el gestor de memoria recibe una `IPoliticaAsignacion` y el planificador trabaja con `Proceso`. Además, declarar `implements` hace que TypeScript avise en compilación si a una clase le falta un método del contrato.

5.2 Encapsulamiento (doble encapsulamiento)

En el primer nivel, todos los atributos son `private` (y `readonly` cuando no cambian), y se leen con `getters`. En el segundo nivel, el estado solo se modifica mediante métodos que validan las reglas del dominio. `Proceso` no tiene `setEstado()`: cada transición es un método con nombre del dominio (admitir, despachar, expulsar, bloquear, terminar) que primero llama al método privado `exigirEstado`, que lanza "Transicion invalida" si el estado de origen no corresponde. Así un proceso no puede ejecutarse sin tener la CPU ni terminar si todavía le queda CPU. El encapsulamiento también protege las colecciones: ninguna clase entrega su lista interna. `GestorMemoria.getMapa()` devuelve copias de los bloques (`VistaBloque`), el planificador devuelve una copia de su cola con `[...colaListos]`, y `Proceso.aVista()` devuelve una copia de sus datos. Los tests de `Proceso`, `BloqueMemoria`, `GestorMemoria` y `Simulador` verifican que modificar lo que devuelven las consultas no altera el estado interno.

5.3 Herencia y composición

Usé herencia solo donde existe una relación real de "es un": `FirstFit`, `BestFit` y `WorstFit` son políticas de asignación y cualquiera puede reemplazar a otra. Las tres hacen lo mismo hasta un punto (filtrar los bloques libres con tamaño suficiente) y solo cambia el criterio de elección. Por eso el filtro está en la clase abstracta `PoliticaAsignacionBase`, en el método protegido `candidatos()`, y cada subclase implementa `elegirBloque()`. En el resto del diseño preferí la composición. `Proceso` no hereda de nada porque no hay tipos

de proceso con comportamiento distinto: lo que cambia es su estado. El Simulador tiene un gestor de memoria y un planificador, pero no es ninguno de ellos, así que los guarda como atributos y les delega el trabajo.

5.4 Polimorfismo

GestorMemoria llama a politica.elegirBloque(bloques, tamaño) sin saber qué política recibió: el mismo mensaje elige el primer hueco, el más chico o el más grande según el objeto que lo recibe. No hay ningún condicional por tipo. Los condicionales del código son validaciones y reglas del dominio, como la prioridad entre finalización, E/S y quantum. En los tests, it.each aplica el mismo caso a las tres políticas sin saber cuál es cada una.

5.5 Principios SOLID y buenas prácticas

Principio	Dónde se aplica
S, responsabilidad única	Las métricas están en CalculadoraMetricas, la memoria en GestorMemoria, la CPU en el planificador y la coordinación en el Simulador.
O, abierto/cerrado	Se puede agregar una política nueva (por ejemplo Next-Fit) creando otra subclase, sin tocar el gestor ni las políticas existentes.
L, sustitución de Liskov	Cualquier política reemplaza a otra; los mismos tests valen para las tres.
I, segregación de interfaces	Cada interfaz es específica de un comportamiento; IPoliticaAsignacion tiene un solo método.
D, inversión de dependencias	GestorMemoria depende de IPoliticaAsignacion y el planificador de IProceso, no de clases concretas. La política se inyecta por constructor.

Otras prácticas: la validación de enteros positivos se concentra en un método privado para no repetirla; los constructores validan sus datos para que ningún objeto exista en un estado inválido; y separo errores de resultados normales. Los datos inválidos y las transiciones prohibidas lanzan un Error con un mensaje claro, mientras que una situación normal, como no tener un hueco suficiente, se informa con un valor de retorno: asignar() devuelve false.

5.6 Estructuras de datos

Elegí estructuras simples, porque la cantidad de procesos y de bloques es chica y lo importante es respetar el orden que pide cada regla:

- Map<number, Proceso> en Simulador: guarda todos los procesos por PID. Con has() detecta un PID duplicado y con get() encuentra un proceso sin recorrer una lista. Además conserva el orden de registro, que es el que usa la admisión.
- Arreglo de BloqueMemoria en GestorMemoria: está ordenado por dirección, que es lo que necesitan First-Fit y el desempate por menor dirección. Al dividir, el sobrante se

inserta con splice justo después del bloque elegido, y al liberar, los vecinos están en las posiciones índice - 1 e índice + 1.

- Arreglo usado como cola FIFO en PlanificadorRoundRobin: push agrega al final y shift saca del principio, que es el orden de llegada de Round Robin.
- Arreglos de pendientes, bloqueados y terminados en Simulador: separan a los procesos según dónde esperan. Se recorren sobre una copia ([...pendientes]) porque dentro del recorrido se quitan elementos con splice.

Ninguna consulta devuelve estas colecciones internas: se devuelven copias o arreglos de PID.

6. Política de memoria utilizada

Usé asignación contigua con particiones dinámicas: cada proceso ocupa un único bloque de direcciones seguidas, del tamaño exacto que pidió. La memoria es una lista de BloqueMemoria ordenada por dirección; cada bloque guarda su inicio, su tamaño y el PID del proceso que lo ocupa, o null si está libre. Guardo el PID y no el objeto Proceso para que la memoria no dependa de esa clase. Al crearse hay un único bloque libre del tamaño total.

Implementé las tres políticas: First-Fit elige el primer bloque suficiente por dirección, Best-Fit el suficiente más chico y Worst-Fit el suficiente más grande. Ante empate, Best-Fit y Worst-Fit eligen la menor dirección, porque recorren los candidatos en orden y la comparación es estricta. La política se elige al configurar, por ejemplo `new GestorMemoria(1024, new BestFit())`. Si el bloque elegido es más grande que lo pedido, se divide: el proceso ocupa la parte inicial y el sobrante queda libre. Una asignación exacta no genera bloques de tamaño cero. Si ningún bloque alcanza, `asignar()` devuelve false sin modificar nada, aunque la suma de memoria libre alcance: ese es el caso de la fragmentación externa.

Al liberar, GestorMemoria marca el bloque libre y llama a `fusionarVecinos()`, que primero absorbe al vecino derecho si está libre y después se deja absorber por el izquierdo. La coalescencia no mueve bloques ocupados, por lo que no es compactación.

6.1 Comparación de las políticas

Como implementé las tres, las comparé con un ejercicio clásico de asignación de memoria: huecos libres de 100, 500, 200, 300 y 600 KB separados por bloques ocupados, y pedidos de 212, 417, 112 y 426 KB en ese orden. Está automatizado en `ComparacionPolíticas.test.ts`.

Política	212 KB	417 KB	112 KB	426 KB	Huecos que quedan (KB)
First-Fit	hueco de 500	hueco de 600	resto de 288	espera	100, 176, 200, 300, 183
Best-Fit	hueco de 300	hueco de 500	hueco de 200	hueco de 600	100, 83, 88, 88, 174
Worst-Fit	hueco de 600	hueco de 500	resto de 388	espera	100, 83, 200, 300, 276

Best-Fit es la única que ubica los cuatro pedidos, porque reserva los huecos grandes para los pedidos grandes. First-Fit y Worst-Fit gastan el hueco de 600 KB antes de que llegue el de 426 KB, que queda esperando aunque haya 959 KB libres en total. La contracara de Best-Fit es que deja huecos muy chicos (83 y 88 KB) que difícilmente se puedan usar después. Ninguna

política es mejor en todos los casos: depende del orden y del tamaño de los pedidos.

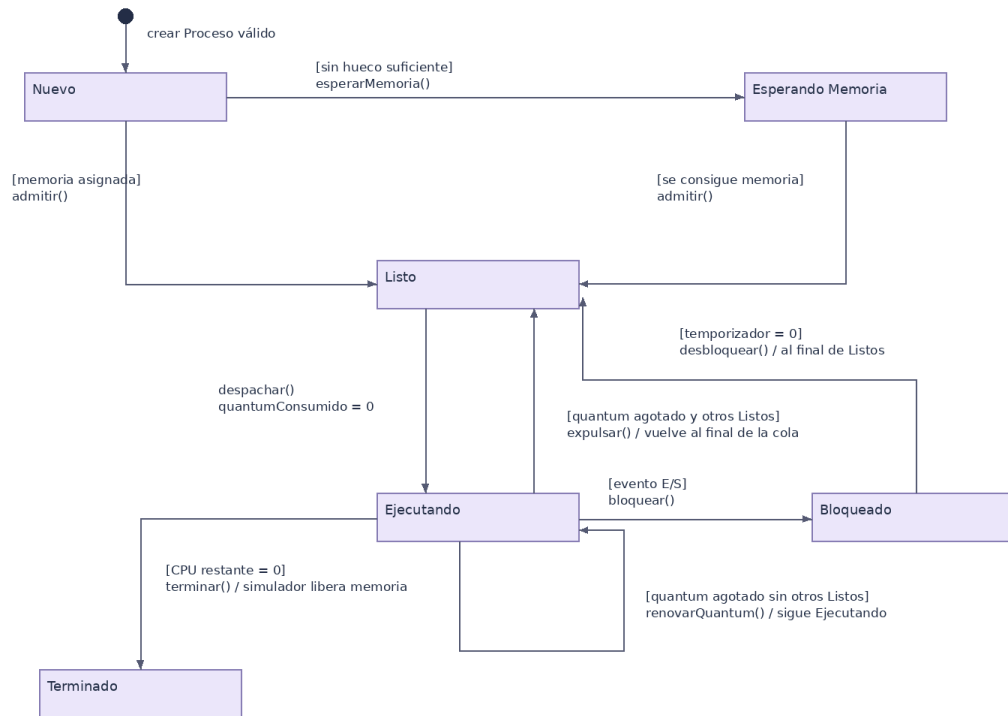
7. Estados de los procesos y orden de ejecución de cada tick

Desde	Hacia	Método	Cuándo ocurre
Nuevo	Listo	admitir()	Hay un hueco suficiente en la admisión
Nuevo	Esperando Memoria	esperarMemoria()	Ningún hueco alcanza
Esperando Memoria	Listo	admitir()	Se liberó memoria y el reintento tuvo éxito
Listo	Ejecutando	despachar()	La CPU está libre y es el primero de la cola; reinicia el quantum
Ejecutando	Ejecutando	renovarQuantum()	Agotó el quantum y no hay otros Listos
Ejecutando	Listo	expulsar()	Agotó el quantum y hay otros Listos
Ejecutando	Bloqueado	bloquear()	Se disparó su evento de E/S; conserva la memoria
Bloqueado	Listo	desbloquear()	El temporizador de bloqueo llegó a 0
Ejecutando	Terminado	terminar()	La CPU restante llegó a 0

Ningún método saca a un proceso del estado Terminado, así que nunca vuelve a las colas.

04 · Estados del proceso

RF03 / RF07 / RF08 · Transiciones durante la coordinación del simulador · Código: 221c194



Cada ejecutarTick() consume una unidad de CPU sin cambiar por sí solo el estado.
Prioridad del planificador: finalización > bloqueo por E/S > quantum. Terminado no vuelve a las colas.

Cada llamada a Simulador.avanzarTick() avanza exactamente un tick, siempre en el mismo orden (RF06):

1. **Admisión:** recorre en orden de registro los procesos pendientes e intenta asignarles memoria. Los que entran pasan a Listo y van al final de la cola; los que no, quedan Esperando Memoria sin impedir que otros sí entren.
2. **Bloqueados:** descuenta un tick a cada bloqueado; al llegar a 0 vuelve al final de la cola de Listos y puede ser despachado en ese mismo tick.
3. **CPU:** el planificador despacha si la CPU está libre y ejecuta una unidad, resolviendo con esta prioridad: si no le queda CPU, termina y se libera su memoria; si se disparó la E/S, se bloquea; si agotó el quantum, se expulsa (si hay otros Listos) o renueva el quantum (si está solo).
4. **Reloj y métricas:** el reloj avanza uno y se recalculan las métricas.

No hay temporizadores, hilos ni aleatoriedad, y como máximo un proceso consume CPU por tick, así que la simulación es determinista. La memoria que libera un proceso al terminar recién se ofrece en la admisión del tick siguiente.

Cambios de contexto: según la convención de la consigna, cuento uno por expulsión por quantum con otros Listos y uno por bloqueo de E/S. No cuento el despacho inicial, la finalización ni la renovación de quantum.

Eventos de E/S: se programan desde los tests indicando después de cuántos ticks de CPU se disparan y cuánto duran. Los dos valores deben ser enteros positivos y el disparo debe ocurrir después de la CPU ya consumida y antes de terminar. Esta validación garantiza por

construcción que la finalización tenga prioridad sobre el bloqueo.

8. Métricas

Métrica	Cálculo
Ocupación de memoria	$100 \times \text{memoria ocupada} / \text{memoria total}$
Utilización de CPU	$100 \times \text{ticks con CPU ocupada} / \text{ticks transcurridos}$; 0% en el tick 0
Cambios de contexto	Contador acumulado según la convención de la sección 7
Memoria libre total	Suma de los bloques libres
Mayor bloque libre	Máximo tamaño libre; 0 si no hay
Fragmentación externa	$100 \times (1 - \text{mayor bloque libre} / \text{memoria libre total})$; 0% si no hay memoria libre

9. Matriz de trazabilidad (RF01 a RF10)

Requerimiento	Clases y métodos	Tests
RF01 Configurar e iniciar	Simulador (constructor), GestorMemoria (constructor), PlanificadorRoundRobin (constructor)	GestorMemoria.test.ts "Inicio (RF01)"; PlanificadorRoundRobin.test.ts; Simulador.test.ts
RF02 Registrar y consultar procesos	Proceso (constructor, getters, aVista); Simulador (registrarProceso, getProceso)	Proceso.test.ts "Registro y validacion (RF02)"; Simulador.test.ts
RF03 Estados y admisión	EstadoProceso; Proceso (esperarMemoria, admitir, exigirEstado); Simulador (fase de admisión)	Proceso.test.ts "Estados y admision (RF03)"; Simulador.test.ts
RF04 Asignar memoria contigua	BloqueMemoria (dividir, asignar); FirstFit, BestFit, WorstFit (elegirBloque); GestorMemoria (asignar)	BloqueMemoria.test.ts; PoliticasAsignacion.test.ts; GestorMemoria.test.ts "Asignacion (RF04)"; ComparacionPoliticas.test.ts
RF05 Liberar y coalescencia	BloqueMemoria (liberar, absorber); GestorMemoria (liberar, fusionarVecinos)	BloqueMemoria.test.ts "Fusion (RF05)"; GestorMemoria.test.ts "Liberacion y coalescencia (RF05)"
RF06 Tick determinista	Simulador (avanzarTick y sus fases)	SimuladorTicks.test.ts "Orden de fases del tick (RF06)"
RF07 Round Robin	PlanificadorRoundRobin (encolar, ejecutarTick); Proceso (despachar, ejecutarTick, renovarQuantum, expulsar, terminar)	PlanificadorRoundRobin.test.ts (incluye P1, P1, P2, P2, P1 con un cambio de contexto); Proceso.test.ts
RF08 Entrada/Salida	Proceso (programarES,	Proceso.test.ts "Entrada/Salida

	debeBloquearse, bloquear, avanzarBloqueo, desbloquear); PlanificadorRoundRobin (prioridad del bloqueo); Simulador (fase de bloqueados)	(RF08)"; PlanificadorRoundRobin.test.ts
RF09 Métricas	CalculadoraMetricas (calcular); GestorMemoria (getMemoriaLibreTotal, getMayorBloqueLibre, getMemoriaOcupada)	CalculadoraMetricas.test.ts; GestorMemoria.test.ts "Datos para metricas (RF09)"
RF10 Consultar el estado	Simulador (getTick, getProcesoEnCpu, getColaListos, getEsperandoMemoria, getBloqueados, getTerminados, getMapaMemoria, getMetricas)	Simulador.test.ts; GestorMemoria.test.ts "el mapa devuelto es una copia"

10. Resultados de las pruebas y cobertura

82 tests en 9 archivos, todos aprobados. La consigna exige verificar cada RF con casos normales, límites y errores, y superar el 90% de cobertura de líneas. Por eso escribí los tests necesarios para cada requerimiento y para los casos mínimos, sin variantes repetidas.

Archivo de test	Tests	Qué verifica
procesos/Proceso.test.ts	19	Creación y validación, transiciones válidas e inválidas, CPU, quantum, E/S y vista de solo lectura
memoria/BloqueMemoria.test.ts	11	Creación, asignar, liberar, dividir sin bloques de tamaño cero, fusión y vista
memoria/PolíticasAsignacion.test.ts	7	Elección de cada política, empates y prueba polimórfica común
memoria/GestorMemoria.test.ts	19	Estado inicial, división, ajuste exacto, fallo sin cambios, coalescencia, invariantes y datos para métricas
memoria/ComparacionPolíticas.test.ts	3	Las tres políticas con la misma carga
planificacion/PlanificadorRoundRobin.test.ts	6	Rotación con un cambio de contexto, renovación de quantum, finalización en el límite, prioridad del bloqueo, CPU ociosa y validaciones

DNI: 43746559

Ingeniería en Sistemas de Información · Universidad de la Cuenca del Plata

AE2 06/10/2026

simulador/CalculadoraMetricas.4 test.ts	4	Tick 0, fórmulas, fragmentación del 25% con huecos de 100 y 300 KB y memoria llena
simulador/Simulador.test.ts y simulador/SimuladorTicks.test.ts	8 + 5	Configuración, registro, admisión y consultas (8); orden del tick, E/S y escenario de la cátedra del main.py (5)

Todos los tests siguen el patrón de preparar, ejecutar y verificar, y están marcados con el RF que cubren. Además de los casos que salen bien, verifico con `toThrow()` que se rechace lo que no corresponde, siempre con el mensaje esperado, para que un test no pase por una razón equivocada.

Para medir la cobertura uso Vitest con el proveedor v8. En `vitest.config.ts` configuré include: `['src/**/*.ts']`, así que se miden todos los archivos de lógica aunque algún test no los importe, y un umbral de líneas de 90.01, porque la consigna exige estrictamente más de 90%. El reporte se regenera con `npm ci` y después `npm run coverage`.

% Coverage report from v8					
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
...es	98.93	99.17	98	99.6	
src	0	0	0	0	
...	0	0	0	0	
...a	100	100	100	100	
...	100	100	100	100	
...	100	100	100	100	
...	100	100	100	100	
...	100	100	100	100	
...	0	0	0	0	
...	0	0	0	0	
...	0	0	0	0	
...	100	100	100	100	
...	100	100	100	100	
...n	100	100	100	100	
...	100	100	100	100	
...	100	100	100	100	
...s	100	100	100	100	
...	100	100	100	100	
...	0	0	0	0	
...	100	100	100	100	
...r	96.25	96.66	93.1	98.59	
...	100	100	100	100	
...	0	0	0	0	
...	0	0	0	0	
...	0	0	0	0	
...	95.89	96.15	92.3	98.48	136

11. Análisis de fragmentación externa

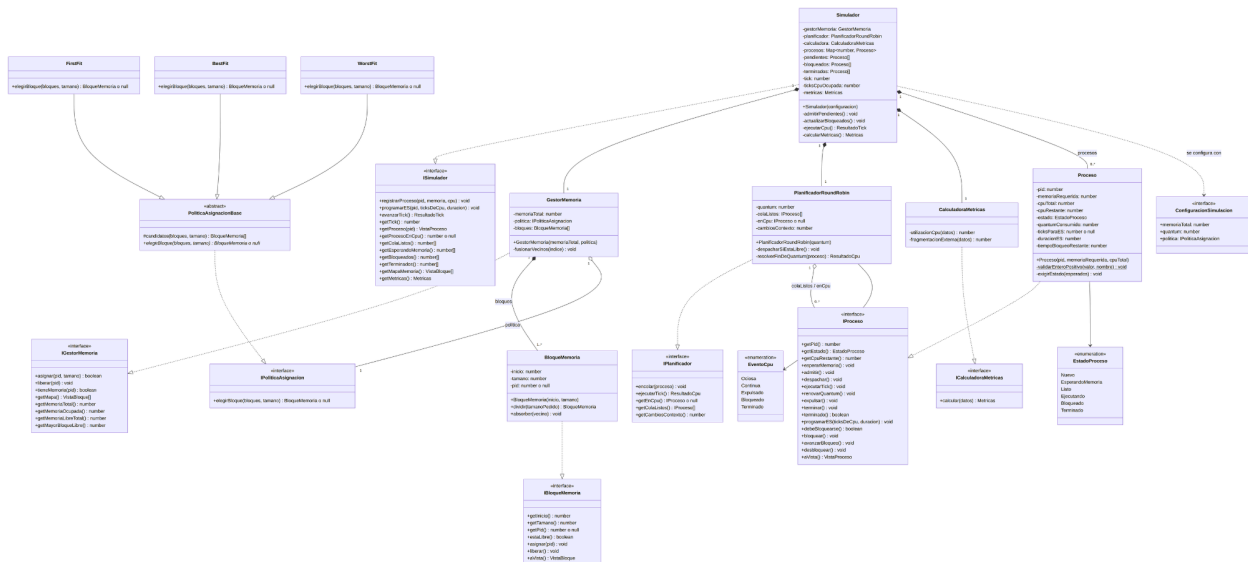
La fragmentación externa aparece cuando la memoria libre total alcanzaría para un proceso, pero está repartida en huecos separados y ninguno es lo bastante grande. Se mide con $F = 100 \times (1 - \text{mayor bloque libre} / \text{memoria libre total})$, y vale 0% si no hay memoria libre.

La comprobé con el caso mínimo de la consigna: con huecos no contiguos de 100 y 300 KB, la memoria libre total es 400 KB, el mayor hueco 300 KB y la fragmentación $100 \times (1 - 300/400) =$

25%. Con la memoria llena da 0%.

En la comparación de la sección 6.1 se ve el caso típico: con First-Fit y Worst-Fit quedan 959 KB libres pero el mayor hueco es de 300 KB, y el pedido de 426 KB no entra. En el escenario de referencia de la cátedra (1024 KB, First-Fit, quantum 2) la fragmentación sube a 27,01% en el tick 8, con 274 KB libres en dos huecos de 200 y 74 KB, y baja a 11,86% en el tick 9, cuando la coalescencia une el bloque liberado con el hueco vecino. Lo verifica el test "Escenario de referencia de la cátedra". Como el simulador no compacta la memoria, los huecos no se juntan moviendo bloques ocupados: la fragmentación baja cuando la coalescencia une huecos vecinos al liberar memoria, o cuando una asignación ocupa alguno de los huecos más chicos.

12. Diagrama de clases UML



La versión editable está en docs/drawio/clases.md, y GitHub la muestra dibujada.

13. Diagramas de secuencia

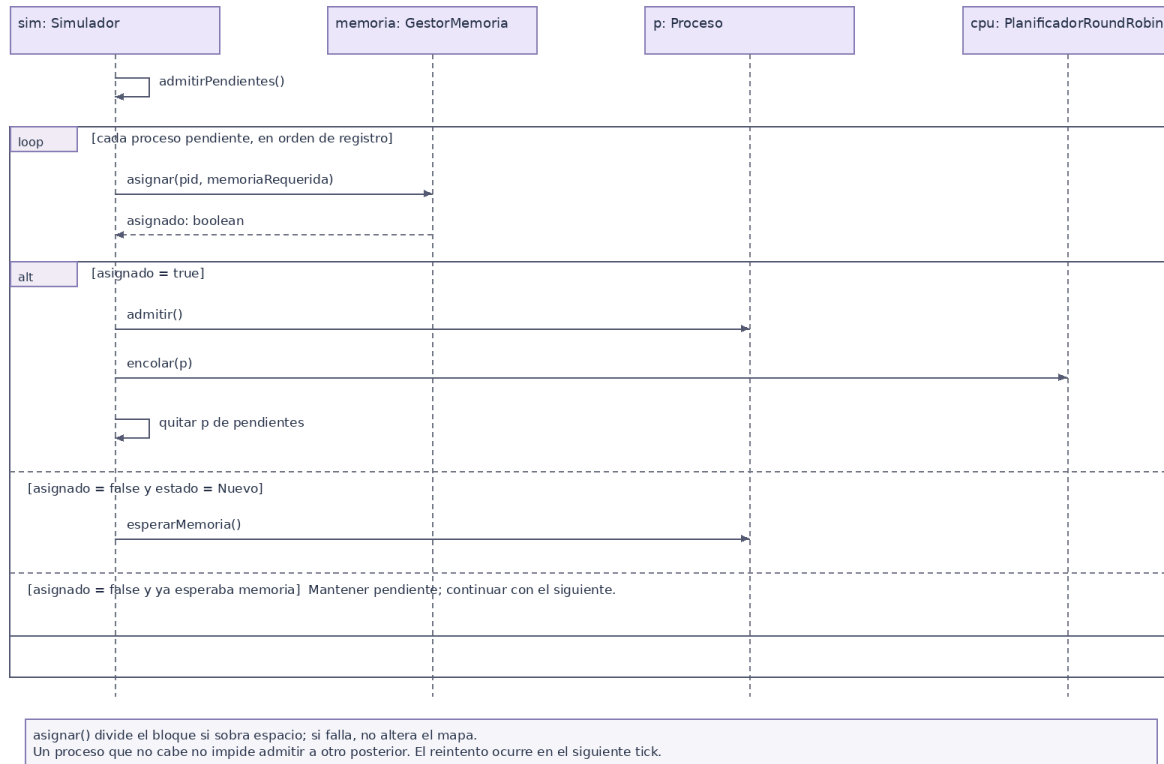
Elegí tres escenarios de requerimientos distintos. Como la biblioteca no tiene main, cada escenario se dispara desde los tests; los diagramas empiezan directamente en los objetos del simulador, en la fase del tick que representan.

13.1 Admisión y asignación de memoria (RF03 y RF04)

El Simulador recorre los pendientes y le pide al GestorMemoria asignar(pid, tamaño). Dentro de asignar(), el gestor le pide un bloque a la política con elegirBloque(), lo divide con dividir() si sobra espacio, lo asigna y devuelve true (el diagrama resume esos pasos internos en la respuesta booleana), y el Simulador llama a admitir() y encola el proceso en el planificador. En el caso alternativo (fragmento alt), si ningún hueco alcanza, asignar() devuelve false sin modificar la memoria y el proceso pasa a Esperando Memoria.

01 · Admisión y asignación de memoria

RF03 / RF04 · Fase 1 de avanzarTick() · Código revisado: 221c194

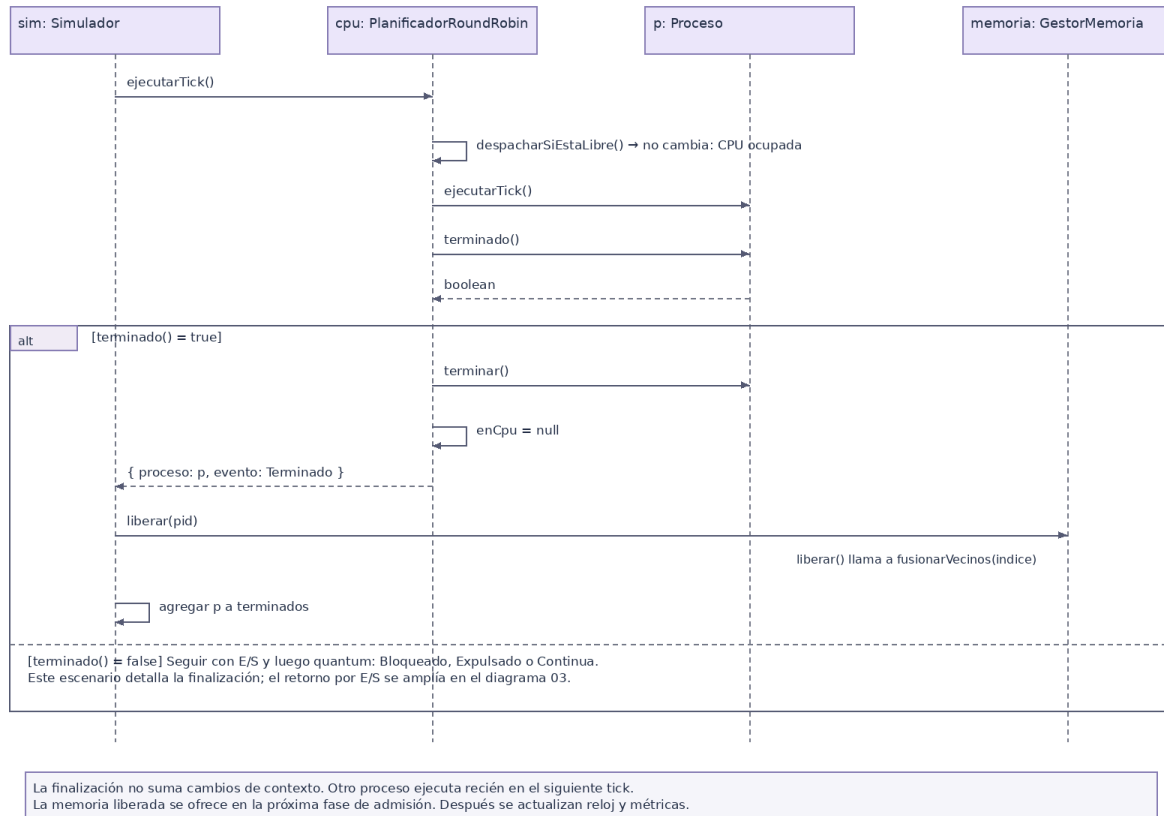


13.2 Un tick de Round Robin con finalización y coalescencia (RF07 y RF05)

El planificador despacha si la CPU está libre y ejecuta un tick. Si el proceso terminó, el Simulador libera su memoria y el gestor fusiona los vecinos libres; si agotó el quantum con otros Listos, se expulsa y suma un cambio de contexto; si está solo, renueva el quantum. El diagrama detalla la finalización y resume las otras alternativas en una nota.

02 · Ejecución de CPU y finalización

RF07 / RF05 · Detalle de la fase 3; proceso ya en CPU al comenzar · Código: 221c194

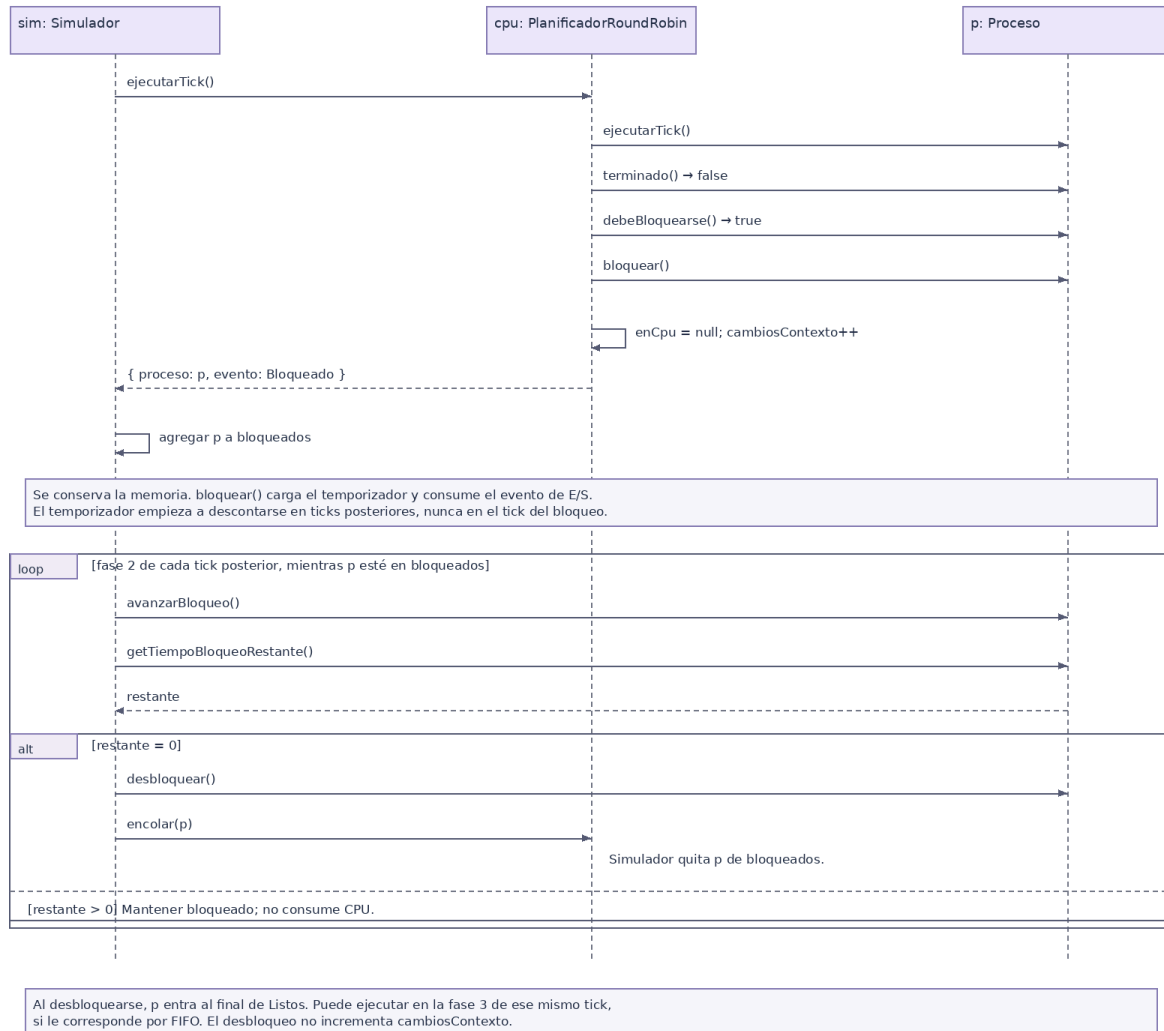


13.3 Bloqueo por E/S y retorno a Listos (RF08)

Al consumir los ticks del evento, debeBloquearse() devuelve true y el proceso se bloquea, liberando la CPU. En los ticks siguientes el Simulador descuenta el temporizador y, al llegar a 0, lo desbloquea y lo encola. No aparece ningún mensaje al gestor de memoria, porque el proceso bloqueado conserva su memoria.

03 · Bloqueo y retorno por entrada/salida

RF08 · Evento programado y alcanzado; CPU restante > 0 · Código revisado: 221c194



14. Metodología, control de versiones e integración continua

Desarrollé con TDD: primero escribía el test del comportamiento y comprobaba que fallara, después el código mínimo para que pasara, y con todo en verde hacía el commit. Construí la biblioteca de abajo hacia arriba: Proceso, después bloques, políticas y gestor de memoria, luego el planificador, las métricas y por último el Simulador. Así pude probar cada clase por separado antes de combinarlas.

Hice un commit por requerimiento o por clase, con el RF en el mensaje, para que el historial sirva como registro del desarrollo. Algunos ejemplos: "test de RF02: registro y validación de proceso", "RF07: transiciones y ejecución de Proceso", "RF08: Simular Entrada y Salida",

DNI: 43746559

Ingeniería en Sistemas de Información · Universidad de la Cuenca del Plata

AE2 06/10/2026

"RF04: Bloque Memoria y Politicas", "RF04 (2da parte) y RF05: Gestor de memoria", "RF04: comparacion de politicas" y "RF03: estados de admision y vista de solo lectura en Proceso". Los últimos fueron "defini contratos y configuracion del simulador", "RF01-RF02-RF03-RF10 Simulador que coordina memoria y CPU", "RF03-RF09 verifica simulacion por ticks", "punto de entrada de la biblioteca", "README con instalacion, comandos y diseño" y uno por cada diagrama. La versión entregada está marcada con el tag v1.0-AE2.

Configuré GitHub Actions con `.github/workflows/ci.yml`. En cada push, en una máquina Ubuntu limpia, instala las dependencias con `npm ci`, verifica los tipos con `npm run typecheck`, ejecuta los tests con cobertura con `npm run coverage`, publica la tabla de cobertura en el resumen del job y guarda el reporte HTML como artefacto.

Vitest Test Report			
Summary			
• Test Files:  9 passes · 9 total • Test Results:  82 passes · 82 total			
Coverage Report			
Status	Category	Percentage	Covered / Total
	Lines	99.6% ( 90%)	249 / 250
	Statements	98.93%	279 / 282
	Functions	98%	98 / 100
	Branches	99.17%	120 / 121

15. Dificultades y aprendizajes

La lección más importante fue un "verde falso": un test de validación pasaba aunque la clase estaba rota, porque `toThrow()` sin argumentos acepta cualquier error. Desde entonces verifico el mensaje exacto. Un test en verde no alcanza si no falla por el motivo correcto.

Tuve varios problemas de entorno en Windows: Git y Node no se reconocían en PowerShell, los comandos de Linux no funcionaban, y al pegar comandos en la terminal se corrompieron archivos de configuración. Lo resolví trabajando con Git Bash y creando los archivos directamente en VS Code. También trabajé en dos carpetas distintas sin darme cuenta, y tuve que dejar una sola conectada a GitHub.

Los imports fallaban porque los nombres de archivo no coincidían en mayúsculas con los de las clases. Windows no lo detecta, pero GitHub Actions corre en Linux y sí. Además, el workflow fallaba aunque los tests pasaban en mi computadora: el paso que publica la cobertura necesitaba el reporte `json-summary`, que mi configuración no generaba. Aprendí que la integración continua detecta problemas que en mi máquina no se ven.

Al agregar `GestorMemoria` fallaron varios tests porque `BloqueMemoria` todavía no tenía `absorber()` ni `aVista()`. El error de una clase puede venir de la clase de la que depende, y declarar `implements` con la interfaz ayuda a detectarlo en compilación. También tuve problemas

de llaves desbalanceadas al pegar métodos dentro de una clase.

A mitad del trabajo se me quedó sin carga la notebook y tuve que seguir a la mañana desde la computadora del trabajo: instalé Git y Node, cloné el repositorio, configuré mi usuario solo para ese repositorio (sin --global) y al terminar borré mis credenciales.

Por último, tenía dudas sobre el uso de if. Entendí que la consigna prohíbe depender de condicionales por tipo concreto, que resolví con polimorfismo, y que los if de validación y de reglas del dominio son necesarios.

Uso de herramientas de IA: usé Claude (Anthropic) como guía para armar el proyecto paso a paso, entender los errores, revisar que el diseño respetara la consigna y para revisar el informe. GitHub Copilot lo usé en el editor para corregir errores de tipeo y llaves que me quedaban sin cerrar y que me daban errores que al principio no entendía. Cada cambio lo probé con npm test y npm run typecheck antes de subirlo, y repasé cada clase para poder explicarla y modificarla en la defensa.

16. Conclusiones

Construí una biblioteca orientada a objetos que simula procesos, memoria contigua con tres políticas, coalescencia, planificación Round Robin y Entrada/Salida, respetando lo que pide la cátedra: interfaces para los comportamientos, doble encapsulamiento, herencia solo donde hay una relación "es un" real, polimorfismo sin condicionales por tipo y composición en el resto del diseño.

Gracias a las interfaces y a la inyección de la política por constructor pude probar cada clase por separado, comparar las tres políticas con la misma carga y agregar una política nueva sin modificar el gestor. La integración continua confirma en cada push que el código se compila y pasa todas las pruebas en una máquina limpia, no solo en la mía.

Desde Sistemas Operativos, el trabajo me permitió ver en concreto la fragmentación externa: memoria suficiente en total que no sirve porque no es contigua, y cómo la coalescencia la reduce al liberar.

Lo que más me costó fue lograr que las clases colaborarán entre sí y funcionan juntas a partir de los casos de prueba que iba armando. Ahora entiendo mejor cómo separar responsabilidades, cómo pensar los tests antes del código y cómo configurar correctamente Vitest y la cobertura. Lo que más satisfacción me dio fue entender conceptos de Sistemas Operativos, como los estados de un proceso, Round Robin y la fragmentación, al mismo tiempo que hacía funcionar los tests que escribía a partir de lo que iba aprendiendo.

17. Fuentes consultadas

- Cátedra de Paradigmas y Lenguajes de Programación II, UCP. (2026). Consigna de la Actividad de Evaluación 2: Simulador de procesos y memoria con POO.
- Cátedra de Sistemas Operativos, UCP. (2026). Simulador de referencia (main.py) y guía "Deconstruyendo el simulador".
- Documentación oficial de TypeScript: <https://www.typescriptlang.org/docs/handbook/>
- Documentación oficial de Vitest: <https://vitest.dev/guide/>
- Documentación oficial de GitHub Actions: <https://docs.github.com/actions>
- Claude (Anthropic) y GitHub Copilot, asistentes de inteligencia artificial usados como apoyo.

DNI: 43746559

Ingeniería en Sistemas de Información · Universidad de la Cuenca del Plata

AE2 06/10/2026

- **diagrams.net.** (s.f.). *draw.io documentation.* <https://www.drawio.com/doc/>
- **Mermaid.** (s.f.). *Mermaid: Class diagrams and sequence diagrams.*
<https://mermaid.js.org/intro/>
(con esto están hechos el diagrama de clases y su versión editable)
- **Git.** (s.f.). *Git documentation.* <https://git-scm.com/doc>